BDA Week 16 — Community Detection

Final-week topic · Girvan-Newman · Modularity · Spectral Clustering

BDA Final Exam Prep · Comprehensive Tutor

Week 16 · Community Detection in Networks

- **§1 — Beginning Key Notes (Study Compass)**
- **§2 — What Is a "Community"?**
  - **Two flavours of "dense"**
  - **Comparison with classical clustering**
  - **Overlapping vs non-overlapping**
- **§3 — Edge Betweenness**
  - **Why betweenness signals community boundaries**
  - **Brandes' algorithm — BFS-based betweenness**
  - **Worked numerical computation — the slide's 7-node graph**
- **§4 — The Girvan-Newman Algorithm**
  - **Pseudocode**
  - **Why "recompute every step" is non-negotiable**
  - **Complexity**
  - **Dendrogram interpretation**
- **§5 — Modularity** QQ
  - **Reading the formula**
  - **Choosing the best cut from the GN dendrogram**
  - **Equivalent block form**
- **§6 — Spectral Clustering Foundations**
  - **Adjacency matrix** AA
  - **Degree matrix** DD
  - **Graph Laplacian** LL
  - **Three load-bearing properties of** LL
  - **The Fiedler vector**
  - **Cheeger inequality (intuition)**
- **§7 — The Spectral Clustering Algorithm**
  - **2-way spectral clustering**
  - kk**-way spectral clustering**
  - **Why it works (intuition)**
  - **Complexity**

§8 — Comparison: Girvan-Newman vs Spectral Clustering
§9 — Six Worked Numerical Examples
Worked Example 1 — Edge betweenness on a 5-node graph (BFS labelling step-by-step)
Worked Example 2 — One Girvan-Newman iteration, with re-computation
Worked Example 3 — Barbell graph: bridge has highest betweenness
Worked Example 4 — Modularity computation step-by-step
Worked Example 5 — Build L for a 5-node graph; find smallest two eigenvalues
Worked Example 6 — Spectral 2-way cut from a Fiedler vector
§10 — Practice Questions (15)
§11 — Full Worked Answers
§12 — Ending Key Notes (Revision Cards)
§13 — Formula & Algorithm Reference

Week 16 · Community Detection in Networks

Why this PDF matters now. Week 16 is the LAST topic of the syllabus and historically the most-likely-to-appear question on the final, especially because it merges two distinct algorithmic flavours — *iterative graph surgery* (Girvan-Newman) and *spectral linear algebra* (Laplacian eigen-decomposition). The examiner has form: he asks you to BFS-label a small graph by hand and compute edge betweenness, OR write down $L=D-AL = D - A$ and find its smallest eigenvalues. Master §3, §4, §6, §7 and you have ~25–35 marks locked in. Modularity (§5) is the bonus question.

§1 — Beginning Key Notes (Study Compass)

These are the ten load-bearing ideas you must walk into the exam owning. Every numerical question on community detection reduces to applying one of them.

1. **A community is a connectivity pattern, not a metric ball.** Unlike k-means clustering (which needs distances between points), community detection works on a graph alone — the only "distance" is path length, and even that is rarely used directly.
2. **Edge betweenness** $b(e)b(e)$ of an edge ee is the number of shortest paths (across all source-target pairs) that pass through ee. **Bridges between communities** carry many shortest paths; **internal edges** carry few. High betweenness = candidate for removal.
3. **Brandes' algorithm** computes betweenness with one BFS per source node: build the BFS DAG, label each node with the number of shortest paths from the source, then propagate "credit" backwards from leaves to root, splitting credit at each branch by the path-count ratio.
4. **Girvan-Newman algorithm:** repeatedly (a) compute betweenness of every edge, (b) remove the highest-betweenness edge, (c) recompute on the surviving graph. Each removal eventually splits a connected component, producing a **dendrogram**.
5. **You must recompute betweenness after every removal.** Skipping this is the most common exam mistake — the examiner deliberately crafts graphs where the second-highest edge changes rank after the first removal.
6. **Modularity** QQ measures how community-like a partition is: $Q=12m\sum ij[Aij−kikj2m]δ(ci,cj)Q = \frac{1}{2m}\sum_{ij}\big[A_{ij} - \frac{k_i k_j}{2m}\big]\delta(c_i, c_j)$. Range: $−1/2≤Q≤1-1/2 \le Q \le 1$. Pick the cut from the GN dendrogram that **maximises** QQ.
7. **Graph Laplacian** $L=D−AL = D - A$ where DD is the diagonal degree matrix and AA is the adjacency matrix. It is symmetric, positive semi-definite, and the all-ones vector $\mathbf{1}$ is always its eigenvector with eigenvalue $λ1=0\lambda_1 = 0$.
8. **Fiedler vector = the eigenvector of the SECOND-smallest eigenvalue** $λ2\lambda_2$**.** Its sign pattern gives a 2-way cut: positive entries → group A, negative entries → group B. The smaller $λ2\lambda_2$ is, the more "splittable" the graph (Cheeger inequality).

9. **Spectral clustering = Laplacian + Fiedler vector + threshold (for 2 clusters)** or **Laplacian + first** kk **eigenvectors + k-means (for** kk **clusters)**. It is fast and gives a globally optimal relaxation of the NP-hard min-cut.
10. **GN vs Spectral.** GN runs in $O(n \cdot m2)$ (one BFS per node per edge removal) — slow on big graphs but produces a full hierarchy. Spectral runs in $O(n3)$ for dense eigen-solve but $O(mm)$ with Lanczos — faster, but only gives one cut at a time.

> **The single biggest exam pattern.** The examiner consistently asks you to either (a) BFS-label a small graph and compute edge betweenness, or (b) write down $L=D-AL = D - A$ for a 5–6 node graph and identify the trivial eigenpair. Master both procedures and 60–70% of the community-detection marks are guaranteed before opening any other topic.

---

§2 — What Is a "Community"?

A **community** in a network is a subset of nodes that are densely interconnected with one another and only sparsely connected to the rest of the graph. Real networks are full of these: friendship circles on Facebook, conferences in NCAA football, functional protein modules, research-collaboration clusters in physics. The community-detection problem is to discover them automatically using the graph structure alone.

Two flavours of "dense"

There are two complementary intuitions for what makes a subset of nodes a community:

- **Connectivity-based.** Inside the community, every pair of nodes is connected by short paths. Across communities, paths are long or pass through a few "bridge" edges. Girvan-Newman exploits exactly this — the bridge edges carry disproportionately many shortest paths and stand out under the betweenness measure.
- **Density-based.** Inside the community, the *fraction* of possible edges that exist is much larger than the graph's overall edge density. A 5-node clique with 10/10 internal edges is a tight community; a 5-node star with only 4 edges is loose. Modularity Q formalises this density-relative-to-baseline view.

Comparison with classical clustering

Classical clustering (k-means, DBSCAN, hierarchical) assumes points live in a metric space — every pair has a real-valued distance. **Graph community detection has no such metric.** You only have $Aij \in \{0,1\}$ $A_{ij} \in \{0, 1\}$: the edge exists or it doesn't. Two non-adjacent nodes are at "infinite distance" in the trivial sense. So the algorithms are fundamentally different:

Aspect	k-means clustering	Community detection
Input	Points in $\mathbb{R}^d$	Graph $G=(V,E)$
Similarity	Euclidean distance	Edge presence + path structure
Cluster shape	Convex (around a centroid)	Whatever the connectivity gives
Need to know k	Yes (must specify)	Often determined by the algorithm
Output	Centroid + assignment	Vertex partition (or hierarchy)

> **EXAM TRAP — communities are not "clusters in the classical sense".** Don't write "use k-means on the adjacency matrix rows" as an answer. The whole point of W16 is that we use *graph-native* algorithms (betweenness, modularity, Laplacian eigenvectors) precisely *because* a node-feature vector is unavailable.

Overlapping vs non-overlapping

The slides also remind us that real-world communities **overlap** — a person can belong to a college friend-group and a workplace friend-group simultaneously. Girvan-Newman and basic spectral clustering produce *partitions* (non-overlapping). Overlap-aware algorithms (BIGCLAM, clique-percolation) are out of scope for the final but worth knowing exist.

---

§3 — Edge Betweenness

Definition. The **betweenness** $b(e)$ of an edge e is the number of shortest paths between all pairs of nodes that pass through e. Formally:

$$b(e)=\sum_{s\neq t\in V}\frac{\sigma_{st}(e)}{\sigma_{st}}$$

where σ_{st} is the total number of shortest paths from s to t, and $\sigma_{st}(e)$ is the number of those that traverse edge e. When all shortest paths between a pair are unique, $\sigma_{st}(e)\in\{0,1\}$ and the formula collapses to "count how many pairs use this edge".

> **EXAM TRAP — edge betweenness vs node betweenness.** *Node* betweenness counts shortest paths through a *vertex*; *edge* betweenness counts paths through an *edge*. Girvan-Newman uses **edge** betweenness. If the question asks you to remove a node, double-check; almost every question is edge-removal.

Why betweenness signals community boundaries

Inside a tight community, between any two members there are usually many short alternative paths — edges share traffic. Between two communities connected by a single bridge, **every** shortest path going from one side to the other must traverse the bridge, so the bridge's betweenness is very large. This is exactly why removing the highest-betweenness edge tends to break a graph into its natural communities.

Brandes' algorithm — BFS-based betweenness

Computing betweenness naively for every edge by enumerating all $O(n2)O(n^2)$ shortest-path pairs is hopeless. **Brandes (2001)** showed how to do it with one BFS per source node, in total $O(n·m)O(n \cdot m)$ time for unweighted graphs:

Per source node s:

1. **BFS from s.** Assign each node v a *level* $d(v)$ = shortest-path distance from s. Build the BFS DAG — edges $(u,v)(u, v)$ with $d(v)=d(u)+1d(v) = d(u) + 1$ point "downward".
2. **Forward pass — count shortest paths.** Set $\sigma(s)=1\sigma(s) = 1$. For each node v in BFS order: $\sigma(v)=\sum u\in parents(v)\sigma(u)\sigma(v) = \sum_{u \in \text{parents}(v)} \sigma(u)$ where "parents" are the predecessors of v in the BFS DAG. $\sigma(v)\sigma(v)$ is the number of shortest paths from s to v.
3. **Backward pass — propagate credit.** Visit nodes in reverse BFS order (leaves first, s last). Initialise every node's "credit" to 1 (every node is a destination at least once). For each node v visited and each of its parents u in the DAG, add to edge $(u,v)(u, v)$ a flow of: $flow(u,v)=credit(v)·\sigma(u)\sigma(v) \text{flow}(u, v) = \text{credit}(v) \cdot \frac{\sigma(u)}{\sigma(v)}$ Then add this flow to u's credit (so credit cascades upward).
4. **Accumulate.** The betweenness of every edge gets a contribution from this source s. Summing over all s and dividing by 2 (each path is counted from both endpoints) gives the final $b(e)b(e)$ — for undirected graphs the factor is 2, but in undergraduate treatments we often just leave the doubled count or normalise consistently.

Worked numerical computation — the slide's 7-node graph

The slides use the following undirected graph (the canonical Girvan-Newman example):

```
    A           D
    |           |\
    |           | \
    B---D       E--G
   /|   |\      |
  A B   E F     F
  ...
```

A cleaner ASCII rendering:

```
A --- B --- D --- E --- G
      |     |     |
      C     F     (E–F edge)
```

Specifically: nodes $\{A,B,C,D,E,F,G\}\{A, B, C, D, E, F, G\}$; edges $\{A-B,B-C,A-C,B-D,D-E,D-F,E-F,E-G,F-G\}\{A\text{-}B, B\text{-}C, A\text{-}C, B\text{-}D, D\text{-}E, D\text{-}F, E\text{-}F, E\text{-}G, F\text{-}G\}$. There are two triangles ($\{A,B,C\}\{A,B,C\}$ and $\{D,E,F\}\{D,E,F\}$) plus the extra connection G to $\{E,F\}\{E,F\}$, all glued together by the single bridge $B-DB\text{-}D$. This is a textbook "barbell-like" structure.

Question: what is $b(B-D)b(B\text{-}D)$?

Answer. Every shortest path from any node in {A,B,C}\{A, B, C\} to any node in {D,E,F,G}\{D, E, F, G\} must traverse B-DB\text{-}D. There are 3×4=123 \times 4 = 12 such pairs, and each has its single shortest path crossing B-DB\text{-}D. Therefore b(B-D)=12b(B\text{-}D) = 12. (The slide's answer.)

For comparison: b(A-B)b(A\text{-}B) carries only shortest paths from AA to nodes outside {A}\{A\} that go through the AA-BB link rather than the AA-CC link. Because AA has another route via CC, half the shortest paths go via AA-BB and half via AA-CC — the calculation is more delicate. We work this out in §9 WE 1.

---

§4 — The Girvan-Newman Algorithm

Girvan-Newman (Phys. Rev. E 2002) is the most influential community-detection algorithm. It is a **divisive hierarchical clustering** method, meaning it starts with the whole graph as one cluster and progressively splits it.

Pseudocode

Input. Undirected graph G=(V,E)G = (V, E).

Output. A hierarchical decomposition (dendrogram) of GG into nested communities.

```
 WHILE E is non-empty:
    Step 1.  Compute b(e) for every edge e ∈ E using Brandes' algorithm.
    Step 2.  Identify e* = argmax_{e ∈ E} b(e).
    Step 3.  Remove e* from E.
    Step 4.  If removing e* increased the number of connected components,
             record this split in the dendrogram.
RETURN  the dendrogram.
```

To get a flat partition with kk communities, walk the dendrogram and stop when there are exactly kk connected components. To pick the *best* number of communities, compute modularity QQ at every level and choose the level with the maximum QQ — see §5.

Why "recompute every step" is non-negotiable

After you remove an edge, the shortest-path structure of the graph changes — paths that used the removed edge are rerouted. The previous betweenness scores are *invalid*. The slide explicitly says **"Need to re-compute betweenness at every step"** for exactly this reason.

A frequent (wrong) shortcut is to compute betweenness once, sort edges in descending order, and remove them in that fixed order. This typically picks the wrong second edge — for instance, after removing a bridge between communities, an edge that was previously medium-importance can suddenly become the critical connector for a within-community substructure.

Complexity

Each BFS is O(n+m)O(n + m). Brandes' algorithm runs nn BFSes, total O(nm)O(nm). The Girvan-Newman algorithm removes mm edges (worst case), and after each removal must rerun Brandes. So the total cost is O(nm2)O(n m^2) — feasible for graphs of a few thousand edges, painfully slow for millions. This is the main reason the spectral approach (§7) is preferred at scale.

Dendrogram interpretation

As edges are removed, the graph shatters in a stylised way: first the bridge edges between major communities go, splitting the graph into 2; then the bridges between sub-communities go, splitting each major piece into 2; and so on. Plotting "number of edges removed" on the x-axis vs the resulting "component structure" on the y-axis gives a tree (dendrogram) where:

- Cuts near the **root** = top-level communities (few, large).
- Cuts near the **leaves** = sub-communities (many, small).
- The user picks a horizontal slice at the level whose modularity is highest.

---

§5 — Modularity

Once you have a candidate partition $C=\{C1,C2,\ldots,Ck\}$ (from any source — Girvan-Newman dendrogram, spectral, by hand), how do you score it? The standard answer is **modularity**, introduced by Newman and Girvan:

$Q(C)=12m\sum i,j[Aij-kikj2m]\delta(ci,cj)$ Q(C) = \frac{1}{2m} \sum_{i, j} \left[A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j)

where:

- mm = total number of edges, so $2m2m$ = sum of all degrees.
- $AijA_{ij}$ = adjacency entry (1 if edge, else 0).
- kik_i = degree of node ii.
- $\delta(ci,cj)=1\delta(c_i, c_j) = 1$ if ii and jj are in the same community, else 0.

Reading the formula

The bracket $Aij-kikj/(2m)A_{ij} - k_i k_j / (2m)$ is **observed minus expected**:

- $AijA_{ij}$ is whether nodes ii and jj are *actually* connected.
- $kikj2m\frac{k_i k_j}{2m}$ is the *expected* number of edges between ii and jj in a random graph that preserves each node's degree (the **configuration null model**). Loosely: in a random rewiring, node ii has kik_i "stub" edge-ends and the chance that a particular stub of ii attaches to one of jj's kjk_j stubs is $kj/(2m)k_j / (2m)$, so the expected count is $ki\cdot kj/(2m)k_i \cdot k_j / (2m)$.

So $Aij-kikj/(2m)A_{ij} - k_i k_j/(2m)$ is positive when ii and jj are connected more than expected (suggesting they share a community), and negative when they are connected less than expected. Multiplying by $\delta(ci,cj)\delta(c_i, c_j)$ keeps only same-community pairs. Summing and normalising by $2m2m$ gives a value in $[-1/2,1][-1/2, 1]$:

- $Q>0Q > 0$: more intra-community edges than chance — the partition is meaningful.
- $Q\approx 0.3Q \approx 0.3$–$0.70.7$: typical "good" community structure in real networks.
- $Q<0Q < 0$: actively worse than random — your partition is junk.

Choosing the best cut from the GN dendrogram

After every Girvan-Newman edge removal, evaluate Q for the current partition (each connected component is one community). Plot Q vs the number of removed edges. Pick the partition at the **peak** of the Q curve — that is the GN-recommended community structure.

> **EXAM TRAP — modularity uses degrees, not out-degrees.** Modularity is defined for **undirected** graphs. k_i is the (undirected) degree of node i, summed once per neighbour. If you accidentally use directed out-degree you will get $\sum_i k_i = m$ rather than $2m$, throwing off every term.

Equivalent block form

If you let L_c = number of edges with both endpoints in community c, and D_c = sum of degrees of nodes in community c, modularity simplifies to:

$$Q = \sum_c \left[\frac{L_c}{m} - \left(\frac{D_c}{2m} \right)^2 \right]$$

This is the form you actually use to compute Q by hand — it requires only two numbers per community, not a double sum.

---

§6 — Spectral Clustering Foundations

Spectral clustering is the second major community-detection paradigm. Instead of greedily editing the graph (Girvan-Newman), it builds a single matrix encoding the graph and reads communities off its eigenvectors.

Adjacency matrix A

For an undirected graph on n nodes:

$$A_{ij} = \begin{cases} 1 & \text{if } (i,j) \in E \\ 0 & \text{otherwise} \end{cases}$$

A is **symmetric** ($A^T = A$), so its eigenvalues are real and its eigenvectors are orthogonal (spectral theorem).

Meaning of Ax. The i-th coordinate of Ax is $\sum_j A_{ij} x_j$ = sum of x-values of i's neighbours. Pre-multiplying by A "spreads" each node's value to its neighbours.

Degree matrix D

$$D_{ii} = k_i = \sum_j A_{ij}, \quad D_{ij} = 0 \text{ for } i \neq j$$

D is diagonal with the node degrees on the diagonal.

Graph Laplacian L

$$\boxed{L = D - A}$$

This is the **combinatorial graph Laplacian**. Some books also use the *normalised* Laplacian $L_{\text{norm}} = I - D^{-1/2} A D^{-1/2}$, but the slides for this course use the un-normalised form $L = D - A$.

Concretely, the entries are:

$$L_{ij} = \begin{cases} k_i & \text{if } i = j \\ -1 & \text{if } (i, j) \in E \\ 0 & \text{otherwise} \end{cases}$$

> **EXAM TRAP — $L = D - A$, NOT $A - D$.** Sign matters! With the wrong sign you get a negative-semidefinite matrix and your eigenvalues come out flipped. The mnemonic: **D**iagonal **m**inus the off-diagonal **a**djacency keeps the diagonal **p**ositive.

Three load-bearing properties of L

1. **Symmetric.** $L^T = D^T - A^T = D - A = L$. Therefore eigenvalues are real, eigenvectors are orthogonal.
2. **Positive semi-definite.** For any vector x: $x^T L x = \sum_{(i,j) \in E} (x_i - x_j)^2 \ge 0$. This identity is so important it's worth seeing why:

$$x^T L x = x^T D x - x^T A x = \sum_i k_i x_i^2 - \sum_{ij} A_{ij} x_i x_j = \sum_{(i,j) \in E} (x_i - x_j)^2$$

The last step is a sum-of-squares rewrite using $\sum_i k_i x_i^2 = \sum_{(i,j) \in E} (x_i^2 + x_j^2)$. Because $(x_i - x_j)^2 \ge 0$, we have $x^T L x \ge 0$ — i.e. $L \succeq 0$, all eigenvalues are non-negative.

1. **Zero eigenvalue is trivial.** Substitute $x = \mathbf{1} = (1, 1, \ldots, 1)^T$. Then $(x_i - x_j)^2 = 0$ for every edge, so $x^T L x = 0$. Combined with $L \succeq 0$, this means $\mathbf{1}$ is an eigenvector with eigenvalue $\lambda_1 = 0$.

The slide states this exactly: **"$x = (1, \ldots, 1)$, then $L \cdot x = 0$ and so $\lambda = \lambda_1 = 0$."** Verify by hand: each row of L sums to $k_i - k_i = 0$, so $L \cdot \mathbf{1} = \mathbf{0}$.

The Fiedler vector

Order the eigenvalues $0 = \lambda_1 \le \lambda_2 \le \ldots \le \lambda_n$ with corresponding eigenvectors $v_1, v_2, \ldots, v_n$.

- $\lambda_1 = 0$, $v_1 = \mathbf{1}$ — trivial, useless.
- λ_2 = the **algebraic connectivity** — the smaller it is, the more loosely the graph is connected.
- v_2 = the **Fiedler vector** — its sign pattern reveals the natural 2-way split.

> **EXAM TRAP — Fiedler vector = SECOND-smallest eigenvalue's eigenvector, NOT smallest.** The smallest is the trivial $\mathbf{1}$ vector, which carries no community information. *Always* skip $\lambda_1 = 0$ and take λ_2.

If the graph has c connected components (rather than 1), then L has exactly c eigenvalues equal to 0 — the multiplicity of 0 as an eigenvalue equals the number of connected components. This generalises cleanly: for a graph with two disconnected clusters, there are two zero eigenvalues, and the corresponding two-dimensional eigenspace is spanned by indicator vectors of the components.

Cheeger inequality (intuition)

The **Cheeger constant** $h(G)$ measures how easily the graph can be cut in half:

$$h(G) = \min_S \frac{|\partial S|}{\min(|S|, |V \setminus S|)}$$

where ∂S is the set of edges with one endpoint in S and one outside. The Cheeger inequality bounds λ_2 in terms of $h(G)$:

$$\frac{\lambda_2}{2} \le h(G) \le \sqrt{2 \lambda_2 \cdot k_{\max}}$$

Translation: small λ_2 ⇔ small $h(G)$ ⇔ the graph is easy to cut into roughly equal halves. The Fiedler vector approximately achieves the optimum cut. You will not be asked to prove Cheeger; just remember that λ_2 **measures bottleneck width**.

---

§7 — The Spectral Clustering Algorithm

Putting §6 to work, the algorithm for partitioning a graph into 2 communities is remarkably short:

2-way spectral clustering

1. Build the adjacency matrix A and the degree matrix D.
2. Form the Laplacian $L = D - A$.
3. Compute the eigenvalues and eigenvectors of L. Identify the second-smallest eigenvalue λ_2 and its eigenvector v_2 (the Fiedler vector).
4. **Threshold by sign:**

$$c_i = \begin{cases} A & \text{if } v_{2}[i] \ge 0 \\ B & \text{if } v_{2}[i] < 0 \end{cases}$$

That's it. Every node is now labelled A or B.

Refinement. Instead of strict sign-thresholding, you can choose the threshold to balance community sizes (median split) or to maximise modularity. For unbalanced graphs, sign-thresholding can dump 90% of nodes on one side; modularity-thresholding picks a less skewed cut.

k-way spectral clustering

To partition into $k > 2$ communities:

1. Build L as before.
2. Compute the **first** k **eigenvectors** $v_1, v_2, \ldots, v_k$ (smallest eigenvalues; v_1 is the trivial $\mathbf{1}$ but is included to handle multi-component cases).
3. Form the $n \times k$ matrix $V = [v_1 \mid v_2 \mid \cdots \mid v_k]$. Each row $V_{i,\cdot}$ is a k-dimensional "spectral embedding" of node i.

4. **Run k-means on the rows of V.** Cluster centroids in $\mathbb{R}^k$ map back to communities.

This is the workhorse spectral algorithm — von Luxburg's "A Tutorial on Spectral Clustering" (2007) is the canonical reference.

Why it works (intuition)

For an "ideal" graph that is exactly k disconnected components, the first k eigenvectors of L are precisely the indicator functions of the components. K-means on those rows trivially recovers the components. For a *nearly* disconnected graph (real communities with a few inter-cluster edges), the ideal eigenvectors get perturbed slightly — but small perturbations don't destroy the cluster structure, so k-means on the perturbed rows still works. This is formalised by **Davis-Kahan theorem** (perturbation bounds for eigenvectors), but the proof is out of scope.

Complexity

- **Dense eigen-solve** (n^3): infeasible for $n > 10{,}000$.
- **Lanczos / sparse iterative methods**: ~$O(m \cdot \sqrt{m})$ per eigenvalue for sparse graphs. Computing only the bottom k eigenvalues is much cheaper than the full spectrum.
- **K-means** on the $n \times k$ embedding: $O(n k \cdot \text{iters})$.

Overall, spectral clustering scales to graphs of $\sim 10^6$ nodes routinely; Girvan-Newman tops out around 10^4.

---

§8 — Comparison: Girvan-Newman vs Spectral Clustering

Aspect	Girvan-Newman	Spectral Clustering
Approach	Divisive hierarchical (top-down edit)	Single linear-algebra computation
What it optimises	Indirectly — removes high-betweenness edges	Relaxation of normalised cut (RatioCut)
Output	Full dendrogram (every k from 1 to n)	One partition (or one per k chosen)
Need to know k in advance	No — pick k post-hoc by maximising Q	Yes — must specify number of eigenvectors
Time complexity	$O(nm^2)$	$O(n^3)$ dense; $O(m\sqrt{m})$ sparse Lanczos
Practical scale	$n \lesssim 10{,}000$	$n \lesssim 10^6$ with sparse solvers
Handles overlapping communities	No	No (basic version)
Determinism	Yes, given tie-breaking rule	Yes (eigenvectors), but k-means step is random
Key strength	Interpretability — see *which* edges are bridges	Speed and a principled global objective
Key weakness	Slow; recomputes betweenness m times	Black-box; sensitive to graph noise

Rule of thumb for the exam. If the question stresses *bridges*, *hierarchy*, or *small graph by hand* — answer Girvan-Newman. If the question stresses *Laplacian*, *eigenvector*, or *fast on large graph* — answer Spectral.

§9 — Six Worked Numerical Examples

These are exactly the kind of multi-step traces the examiner sets. Read each one once, then close this document and reproduce it on paper from scratch.

Worked Example 1 — Edge betweenness on a 5-node graph (BFS labelling step-by-step)

Problem. Consider the graph with nodes $\{A, B, C, D, E\}$ and edges $\{A\text{-}B, A\text{-}C, B\text{-}C, B\text{-}D, D\text{-}E\}$. Compute the betweenness of every edge using Brandes' algorithm with source A.

Graph.

```
    A
   / \
  B---C
  |
  D
  |
  E
```

Step 1 — BFS from AA. Levels:

- Level 0: AA.
- Level 1: neighbours of AA → B,CB, C.
- Level 2: new neighbours of {B,C}\{B, C\} → DD (via BB). CC has no new neighbour besides BB which is already labelled.
- Level 3: new neighbours of DD → EE.

```
Level 0:   A
           |\
Level 1:   B  C
           |
Level 2:   D
           |
Level 3:   E
```

The BFS DAG has these directed (parent → child) edges: A→BA \to B, A→CA \to C, B→DB \to D, D→ED \to E. Note that the edge BB-CC is at the *same* BFS level (both level 1), so it is NOT in the BFS DAG.

Step 2 — Forward pass (label nodes with σ\sigma, the number of shortest paths from AA).

- σ(A)=1\sigma(A) = 1.
- σ(B)=σ(A)=1\sigma(B) = \sigma(A) = 1 (only parent is AA).
- σ(C)=σ(A)=1\sigma(C) = \sigma(A) = 1 (only parent is AA).
- σ(D)=σ(B)=1\sigma(D) = \sigma(B) = 1 (only parent in DAG is BB).
- σ(E)=σ(D)=1\sigma(E) = \sigma(D) = 1.

Step 3 — Backward pass (propagate credit). Visit in reverse BFS order: E,D,B,C,AE, D, B, C, A.

Initialise every node's credit to 1 (each node is its own destination once).

- **Node EE (leaf, level 3).** Credit = 1. Single parent DD. Flow on edge D-ED\text{-}E = credit(E)·σ(D)/σ(E)=1·1/1=1\text{credit}(E) \cdot \sigma(D)/\sigma(E) = 1 \cdot 1/1 = 1. Add 1 to DD's credit. Now credit(D)=1+1=2\text{credit}(D) = 1 + 1 = 2.
- **Node DD (level 2).** Credit = 2. Single parent BB. Flow on edge B-DB\text{-}D = 2·1/1=22 \cdot 1/1 = 2. Add 2 to BB's credit. Now credit(B)=1+2=3\text{credit}(B) = 1 + 2 = 3.
- **Node CC (level 1).** Credit = 1. Single parent AA. Flow on edge A-CA\text{-}C = 1·1/1=11 \cdot 1/1 = 1. credit(A)+=1\text{credit}(A) += 1.

- **Node** BB **(level 1).** Credit = 3. Single parent AA. Flow on edge A-BA\text{-}B = 3·1/1=33 \cdot 1/1 = 3. credit(A)+=3\text{credit}(A) += 3.

Step 4 — Edge contributions from source AA**:**

Edge	Contribution from
AA-BB	3
AA-CC	1
BB-DD	2
DD-EE	1
BB-CC	0 (not in DAG)

Step 5 — Repeat for sources B,C,D,EB, C, D, E**.** I'll summarise (you should do at least one more by hand for practice). Doing the same procedure starting from BB, CC, DD, EE and summing all contributions, then dividing by 2 (each path is counted twice in undirected BFS), gives:

Edge	Total betweenness
AA-BB	3.0
AA-CC	3.0
BB-CC	1.0
BB-DD	6.0
DD-EE	4.0

Highest betweenness: edge BB-DD with b=6b = 6. This is the bridge that connects the triangle {A,B,C}\{A, B, C\} to the path {D,E}\{D, E\}, and makes intuitive sense — every shortest path from AA or CC to DD or EE must cross BB-DD. (There are 3×2=63 \times 2 = 6 such pairs and each has unique shortest path through BB-DD.) Removing BB-DD would split the graph into the triangle and the path — the natural community structure.

Worked Example 2 — One Girvan-Newman iteration, with re-computation

Problem. Continuing from WE 1, run one full GN iteration: identify the highest-betweenness edge, remove it, then RE-COMPUTE the betweenness on the remaining graph.

Step 1 — From WE 1, b(B-D)=6b(B\text{-}D) = 6 **is highest. Remove edge** BB**-**DD**.**

Remaining graph:

```
    A
   / \
  B---C       D---E   (two components)
```

Component 1: triangle {A,B,C}\{A, B, C\}. Component 2: path {D,E}\{D, E\}.

Step 2 — Recompute betweenness on the remaining graph. Betweenness is computed *within* each component:

- **Triangle** {A,B,C}\{A, B, C\}**.** For any pair, there are *two* shortest paths of length 1 (the direct edge) plus length-2 paths through the third node. But shortest paths use the direct edge, so each of the three edges has b=1b = 1 from the unique pair (length-1 paths). Specifically, b(A-B)=b(A-C)=b(B-C)=1b(A\text{-}B) = b(A\text{-}C) = b(B\text{-}C) = 1.
- **Path** {D,E}\{D, E\}**.** Single edge D-ED\text{-}E with b=1b = 1 (the unique pair D,ED, E).

Step 3 — Remove the new highest-betweenness edge. All four remaining edges tie at b=1b = 1. Tie-breaking is convention-dependent — say we remove AA-BB. The triangle becomes a path {B,C,A}\{B, C, A\} (still connected through CC).

Step 4 — Recompute again. Now AA-CC and BB-CC each carry the unique shortest path between AA and BB (length 2 via CC): b(A-C)=b(B-C)=1b(A\text{-}C) = b(B\text{-}C) = 1. Edge DD-EE still b=1b=1.

Observation. This is exactly why you must recompute every step: had we sorted edges once and taken the top-2, we might have removed an arbitrary triangle edge before BB-DD. Recomputation guarantees that at every step we attack the current bridge.

Worked Example 3 — Barbell graph: bridge has highest betweenness

Problem. Consider a "barbell" graph: two triangles {1,2,3}\{1, 2, 3\} and {4,5,6}\{4, 5, 6\} connected by a single edge 33-44 (the bridge). Compute the betweenness of edge 33-44 and confirm Girvan-Newman cleanly recovers the two communities.

Graph.

```
    1           4
   / \         / \
  2---3--------4---5
      \       /
       ......       (edge 3–4 is the only connection)

  actually:

  1 --- 2          5 --- 4
   \   /            \   /
     3 ------------- 6
                      wait, let me re-draw cleanly
```

Cleanly: nodes {1,2,3,4,5,6}\{1, 2, 3, 4, 5, 6\}; edges {1-2,1-3,2-3,3-4,4-5,4-6,5-6}\{1\text{-}2, 1\text{-}3, 2\text{-}3, 3\text{-}4, 4\text{-}5, 4\text{-}6, 5\text{-}6\}.

```
     1             5
    |\           /|
    | \         / |
    |  3 ----- 4  |
    | /         \ |
    |/           \|
    2             6
```

Step 1 — Identify the bridge. Edge 33-44 is the *only* path between any node in {1,2,3}\{1, 2, 3\} and any node in {4,5,6}\{4, 5, 6\}. Therefore every shortest path crossing the boundary must traverse 33-44.

Step 2 — Count crossing shortest paths. There are 3×3=93 \times 3 = 9 pairs (u,v)(u, v) with u∈{1,2,3}u \in \{1, 2, 3\} and v∈{4,5,6}v \in \{4, 5, 6\}. Each pair has a unique shortest path passing through 33-44. Therefore b(3-4)=9b(3\text{-}4) = 9.

Step 3 — Compare with internal-triangle edges. Take edge 11-22. It carries the shortest path between 11 and 22 — but there is also an alternative length-2 path 11-33-22. So the unique pair (1,2)(1, 2) contributes 11 shortest path of length 1 entirely on 11-22. Could 11-22 also lie on shortest paths from 11 to other nodes? Shortest path 1→41 \to 4 is 1-3-41\text{-}3\text{-}4 (length 2), not via 22. So 11-22 contributes only on the (1,2)(1, 2) pair → b(1-2)=1b(1\text{-}2) = 1. By symmetry, every triangle edge has b=1b = 1.

Step 4 — Run Girvan-Newman. Highest betweenness: b(3-4)=9b(3\text{-}4) = 9. Remove it. Graph splits into {1,2,3}\{1, 2, 3\} and {4,5,6}\{4, 5, 6\} — the two natural communities. **Done in one iteration.**

Modularity of this partition. Total edges m=7m = 7; degrees: k1=k2=k5=k6=2k_1 = k_2 = k_5 = k_6 = 2, k3=k4=3k_3 = k_4 = 3. Sum of degrees =14=2m= 14 = 2m ✓.

Use the block form: Q=∑c[Lc/m−(Dc/2m)2]Q = \sum_c [L_c/m - (D_c/2m)^2].

- Community {1,2,3}\{1, 2, 3\}: internal edges L1=3L_1 = 3 (the three triangle edges); D1=2+2+3=7D_1 = 2 + 2 + 3 = 7.
- Community {4,5,6}\{4, 5, 6\}: L2=3L_2 = 3; D2=3+2+2=7D_2 = 3 + 2 + 2 = 7.

Q=[37−(714)2]+[37−(714)2]=2·[37−14]=2·(0.4286−0.25)=2·0.1786=0.357 Q = \left[\frac{3}{7} - \left(\frac{7}{14}\right)^2\right] + \left[\frac{3}{7} - \left(\frac{7}{14}\right)^2\right] = 2 \cdot \left[\frac{3}{7} - \frac{1}{4}\right] = 2 \cdot (0.4286 - 0.25) = 2 \cdot 0.1786 = 0.357

Q≈0.36Q \approx 0.36 — strong community structure (recall Q∈[−0.5,1]Q \in [-0.5, 1] and real-world communities typically score 0.30.3–0.70.7).

Worked Example 4 — Modularity computation step-by-step

Problem. A small graph on 4 nodes with edges {1-2,2-3,3-4,4-1,1-3}\{1\text{-}2, 2\text{-}3, 3\text{-}4, 4\text{-}1, 1\text{-}3\} (a 4-cycle plus the diagonal 11-33). Partition: C1={1,2},C2={3,4}C_1 = \{1, 2\}, C_2 = \{3, 4\}. Compute QQ.

Step 1 — basic stats. Edges: 5, so m=5m = 5, 2m=102m = 10. Degrees: k1=3k_1 = 3 (neighbours 2,3,42, 3, 4), k2=2k_2 = 2 (1,31, 3), k3=3k_3 = 3 (2,4,12, 4, 1), k4=2k_4 = 2 (3,13, 1). Check: ∑ki=10=2m\sum k_i = 10 = 2m ✓.

Step 2 — Adjacency matrix.

	1	2	3	4
1	0	1	1	1
2	1	0	1	0
3	1	1	0	1
4	1	0	1	0

Step 3 — block-form modularity.

- Community C1={1,2}C_1 = \{1, 2\}. Internal edges: only {1,2}\{1, 2\} → L1=1L_1 = 1. Degree sum: D1=3+2=5D_1 = 3 + 2 = 5.
- Community C2={3,4}C_2 = \{3, 4\}. Internal edges: only {3,4}\{3, 4\} → L2=1L_2 = 1. Degree sum: D2=3+2=5D_2 = 3 + 2 = 5.

Q=[15−(510)2]+[15−(510)2]=2·(0.2−0.25)=−0.10 Q = \left[\frac{1}{5} - \left(\frac{5}{10}\right)^2\right] + \left[\frac{1}{5} - \left(\frac{5}{10}\right)^2\right] = 2 \cdot (0.2 - 0.25) = -0.10

Q<0Q < 0: this partition is **worse than random**. Inspection: most "high-degree" interaction in this graph is *across* the partition (edges 22-3,33, 3-44 wait, 33-44 is intra-C2C_2… 11-3,13, 1-44 are cross-cut; 22-33 is cross-cut). So we have only 2 intra-community edges out of 5 — the cut is bad. A better partition would be {1,2,3},{4}\{1, 2, 3\}, \{4\} or {1,3},{2,4}\{1, 3\}, \{2, 4\}.

Try partition {1,2,3},{4}\{1, 2, 3\}, \{4\}**.**

- C1={1,2,3}C_1 = \{1, 2, 3\}: internal edges {1-2,2-3,1-3}\{1\text{-}2, 2\text{-}3, 1\text{-}3\} → L1=3L_1 = 3. D1=3+2+3=8D_1 = 3 + 2 + 3 = 8.
- C2={4}C_2 = \{4\}: L2=0L_2 = 0, D2=2D_2 = 2.

Q=[35−(810)2]+[05−(210)2]=(0.6−0.64)+(0−0.04)=−0.04−0.04=−0.08 Q = \left[\frac{3}{5} - \left(\frac{8}{10}\right)^2\right] + \left[\frac{0}{5} - \left(\frac{2}{10}\right)^2\right] = (0.6 - 0.64) + (0 - 0.04) = -0.04 - 0.04 = -0.08

Slightly less bad. The graph is small enough that no partition gives strongly positive QQ — that is itself a useful diagnostic: this graph has no real community structure.

Worked Example 5 — Build LL for a 5-node graph; find smallest two eigenvalues

Problem. Path graph on 5 nodes: 1−2−3−4−51 - 2 - 3 - 4 - 5. Build LL and find λ1,λ2\lambda_1, \lambda_2 and the Fiedler vector.

Step 1 — degrees. k1=1,k2=2,k3=2,k4=2,k5=1k_1 = 1, k_2 = 2, k_3 = 2, k_4 = 2, k_5 = 1.

Step 2 — adjacency matrix AA**.**

A=(0100010100010100010100010) A = \begin{pmatrix} 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 \end{pmatrix}

Step 3 — degree matrix DD**.**

D=diag(1,2,2,2,1) D = \text{diag}(1, 2, 2, 2, 1)

Step 4 — Laplacian L=D−AL = D - A**.**

L=(1−1000−12−1000−12−1000−12−1000−11) L = \begin{pmatrix} 1 & -1 & 0 & 0 & 0 \\ -1 & 2 & -1 & 0 & 0 \\ 0 & -1 & 2 & -1 & 0 \\ 0 & 0 & -1 & 2 & -1 \\ 0 & 0 & 0 & -1 & 1 \end{pmatrix}

Step 5 — verify trivial eigenpair. Compute L·**1**L \cdot \mathbf{1} row-by-row: 1−1=01-1=0, −1+2−1=0-1+2-1=0, 0−1+2−1=00-1+2-1=0, −1+2−1=0-1+2-1=0, −1+1=0-1+1=0. So L·**1**=**0**L \cdot \mathbf{1} = \mathbf{0} ✓ — confirms λ1=0\lambda_1 = 0 with eigenvector v1=**1**v_1 = \mathbf{1}.

Step 6 — find λ2\lambda_2. For a path graph PnP_n on nn nodes the Laplacian eigenvalues have closed form:

λk=2−2cos((k−1)πn),k=1,2,...,n \lambda_k = 2 - 2\cos\left(\frac{(k-1)\pi}{n}\right), \quad k = 1, 2, \ldots, n

For n=5,k=2n = 5, k = 2: λ2=2−2cos(π/5)=2−2(0.809)=2−1.618=0.382\lambda_2 = 2 - 2\cos(\pi/5) = 2 - 2(0.809) = 2 - 1.618 = 0.382.

The corresponding **Fiedler vector** for PnP_n is:

v2[i]=cos((2i−1)π2n),i=1,...,n v_2[i] = \cos\left(\frac{(2i - 1)\pi}{2n}\right), \quad i = 1, \ldots, n

For n=5n = 5:

- v2[1]=cos(π/10)=0.951v_2[1] = \cos(\pi/10) = 0.951
- v2[2]=cos(3π/10)=0.588v_2[2] = \cos(3\pi/10) = 0.588
- v2[3]=cos(5π/10)=0v_2[3] = \cos(5\pi/10) = 0
- v2[4]=cos(7π/10)=−0.588v_2[4] = \cos(7\pi/10) = -0.588
- v2[5]=cos(9π/10)=−0.951v_2[5] = \cos(9\pi/10) = -0.951

(Up to overall sign.) This is a smoothly varying signal that goes positive on the “left half” and negative on the “right half” of the path — the natural 2-cut.

Step 7 — read off the cut. Sign-threshold the Fiedler vector: nodes with v2≥0v_2 \ge 0 are {1,2,3}\{1, 2, 3\}, nodes with v2<0v_2 < 0 are {4,5}\{4, 5\}. (Node 3 is a tie at 0; conventionally assigned to either side.) This cuts the path between nodes 3 and 4 — exactly in the middle, which is the natural splitting point.

Sanity check via xTLxx^T L x. For the cut indicator x=(+1,+1,0,−1,−1)x = (+1, +1, 0, -1, -1), xTLx=∑(i,j)∈E(xi−xj)2=0+1+1+0=2x^T L x = \sum_{(i,j)\in E}(x_i - x_j)^2 = 0 + 1 + 1 + 0 = 2. This is the **cut size** times 2 — and we cut exactly 2 edges (3-4 and conceptually around node 3), which matches the calculation.

Worked Example 6 — Spectral 2-way cut from a Fiedler vector

Problem. Suppose you have computed the Fiedler vector v2=[−0.5,−0.3,+0.2,+0.4,+0.5]v_2 = [-0.5, -0.3, +0.2, +0.4, +0.5] for a 5-node graph. Apply spectral clustering to partition into 2 communities.

Step 1 — sign-threshold each entry.

Node		Sign	Community
1	-0.5	−-	B
2	-0.3	−-	B
3	+0.2	++	A
4	+0.4	++	A
5	+0.5	++	A

Step 2 — read off the partition. Community A={3,4,5}A = \{3, 4, 5\}, Community B={1,2}B = \{1, 2\}.

Step 3 — sanity check. The Fiedler vector entries are arranged in increasing order — node 1 is most extreme on the negative side (most "B-like"), node 5 most extreme on the positive side (most "A-like"). The transition from negative to positive happens between nodes 2 and 3, marking the cut.

Variant — balanced cut. If you wanted exactly 50/50 partition (ignoring graph structure for a moment), you would median-threshold instead: median of {−0.5,−0.3,0.2,0.4,0.5}\{-0.5, -0.3, 0.2, 0.4, 0.5\} is 0.20.2, so split at ≤0.2\le 0.2 vs >0.2> 0.2, giving B={1,2,3}B = \{1, 2, 3\} and A={4,5}A = \{4, 5\}. Sign-thresholding gave {3,4,5}\{3, 4, 5\} vs {1,2}\{1, 2\} — for clean cluster recovery you usually trust the sign threshold; balanced thresholding is a workaround for pathological cases.

---

§10 — Practice Questions (15)

Mix: 6 numerical, 5 conceptual short-answer, 4 MCQ / true-false. Time yourself: ∼6\sim 6 min per numerical, ∼3\sim 3 min per conceptual, ∼1\sim 1 min per MCQ.

Q1 [Numerical · 8 marks]. Consider the graph: nodes {1,2,3,4,5}\{1, 2, 3, 4, 5\}, edges {1-2,2-3,3-4,4-5,1-5} \{1\text{-}2, 2\text{-}3, 3\text{-}4, 4\text{-}5, 1\text{-}5\} (a 5-cycle). (a) Run BFS from node 1 and write down the BFS levels. (b) Compute σ(v)\sigma(v) for each node. (c) Compute the contribution of source 1 to the betweenness of every edge.

Q2 [Numerical · 6 marks]. A barbell graph: nodes {1,2,3,4,5,6,7,8}\{1, 2, 3, 4, 5, 6, 7, 8\}; left clique K4K_4 on {1,2,3,4}\{1, 2, 3, 4\} (all 6 edges), right clique K4K_4 on {5,6,7,8}\{5, 6, 7, 8\}, plus single bridge 44-55. State b(4-5)b(4\text{-}5) and b(1-2)b(1\text{-}2). Which edge does Girvan-Newman remove first?

Q3 [Numerical · 7 marks]. Graph with edges {1-2,1-3,2-3,3-4,4-5,4-6,5-6}\{1\text{-}2, 1\text{-}3, 2\text{-}3, 3\text{-}4, 4\text{-}5, 4\text{-}6, 5\text{-}6\} (two triangles glued at node 4 via edge 33-44). Compute modularity QQ for the partition {1,2,3},{4,5,6}\{1, 2, 3\}, \{4, 5, 6\} using the block formula.

Q4 [Numerical · 8 marks]. A 4-node graph with edges {1-2,2-3,3-4}\{1\text{-}2, 2\text{-}3, 3\text{-}4\} (a path on 4 nodes). (a) Build A,D,LA, D, L. (b) Verify L·1=0L \cdot \mathbf{1} = \mathbf{0} entry-by-entry. (c) Confirm symmetry. (d) Without solving the full eigenproblem, explain why λ2>0\lambda_2 > 0.

Q5 [Numerical · 7 marks]. Given Fiedler vector v2=[+0.6,+0.4,+0.1,−0.2,−0.5,−0.4]v_2 = [+0.6, +0.4, +0.1, -0.2, -0.5, -0.4] for a 6-node graph: (a) Sign-threshold to get the 2-way partition. (b) Median-threshold to get the balanced partition. (c) Comment on which is more reliable.

Q6 [Numerical · 6 marks]. For the 4-cycle {1-2,2-3,3-4,4-1}\{1\text{-}2, 2\text{-}3, 3\text{-}4, 4\text{-}1\}, build LL and compute λ2\lambda_2 using the formula λk=2−2cos(2π(k−1)/n)\lambda_k = 2 - 2\cos(2\pi(k-1)/n) for cycle CnC_n. State the Fiedler vector.

Q7 [Concept · 4 marks]. Define edge betweenness in one sentence. Explain in 2–3 sentences why edges between communities tend to have high betweenness.

Q8 [Concept · 4 marks]. State why the Girvan-Newman algorithm must recompute betweenness after each edge removal. Give a specific example where skipping recomputation would pick the wrong second edge.

Q9 [Concept · 4 marks]. Define modularity QQ. Explain the meaning of the term Aij−kikj/(2m)A_{ij} - k_i k_j /(2m) — what is the configuration null model?

Q10 [Concept · 3 marks]. State the three load-bearing properties of the Laplacian L=D−AL = D - A and give a one-sentence justification of each.

Q11 [Concept · 4 marks]. Why is the Fiedler vector (eigenvector of λ2\lambda_2) the "right" thing to threshold for a 2-way cut, rather than the eigenvector of λ1\lambda_1?

Q12 [MCQ · 2 marks]. For an undirected unweighted graph, the Laplacian L=D−AL = D - A: (A) is symmetric and negative semi-definite (B) is symmetric and positive semi-definite, with smallest eigenvalue 00 (C) has all eigenvalues in [−1,1][-1, 1] (D) is column-stochastic

Q13 [MCQ · 2 marks]. In the Girvan-Newman algorithm, the edge with highest betweenness: (A) is always inside a community (B) is always a bridge between two communities (C) is most likely a bridge between communities (D) carries no shortest paths

Q14 [True/False · 2 marks]. "If a graph has 3 connected components, the Laplacian has exactly 3 eigenvalues equal to 0." Justify.

Q15 [MCQ · 2 marks]. Which procedure does kk-way spectral clustering use after computing the Laplacian's smallest kk eigenvectors? (A) Sign-threshold each row. (B) Run k-means on the rows of the eigenvector matrix. (C) Apply Girvan-Newman to the resulting embedding. (D) Take the median of each row.

---

§11 — Full Worked Answers

A1. (a) BFS levels from 1: - Level 0: {1}\{1\}. - Level 1: {2,5}\{2, 5\} (neighbours of 1). - Level 2: {3,4}\{3, 4\} (33 via 22; 44 via 55).

(b) σ\sigma values. σ(1)=1\sigma(1) = 1; σ(2)=1\sigma(2) = 1 (parent: 1); σ(5)=1\sigma(5) = 1 (parent: 1); σ(3)=1\sigma(3) = 1 (parent: 2 only — there is no shorter path via 44); σ(4)=1\sigma(4) = 1 (parent: 5 only). **Note:** the edge 33-44 is *between* level-2 nodes so not in the BFS DAG.

(c) Backward credit pass from source 1.

- Visit nodes in reverse BFS order: {3,4}\{3, 4\} (level 2), then {2,5}\{2, 5\} (level 1), then {1}\{1\} (level 0).
- Node 3, credit = 1, parent 2 (only). Edge 22-33 flow = 1·1/1=11 \cdot 1/1 = 1. credit(2)=1+1=2\text{credit}(2) = 1 + 1 = 2.
- Node 4, credit = 1, parent 5 (only). Edge 44-55 flow = 1. credit(5)=2\text{credit}(5) = 2.

- Node 2, credit = 2, parent 1. Edge 11-22 flow = 2.
- Node 5, credit = 2, parent 1. Edge 11-55 flow = 2.

Edge betweenness contributions from source 1: b(1-2)+=2b(1\text{-}2) += 2, b(2-3)+=1b(2\text{-}3) += 1, b(3-4)+=0b(3\text{-}4) += 0, b(4-5)+=1b(4\text{-}5) += 1, b(1-5)+=2b(1\text{-}5) += 2.

A2. b(4-5)b(4\text{-}5): edge 44-55 is the only bridge between the two cliques. Every shortest path from any node in {1,2,3,4}\{1, 2, 3, 4\} to any node in {5,6,7,8}\{5, 6, 7, 8\} uses it. There are 4×4=164 \times 4 = 16 such pairs, each with unique shortest path. So b(4-5)=16b(4\text{-}5) = 16.

b(1-2)b(1\text{-}2): this edge lies inside a complete clique K4K_4. The pair (1,2)(1, 2) has multiple shortest paths of length 1 — only one path uses edge 11-22 directly. There are no shortest paths from 11 to other clique nodes that use 11-22 (those go directly via the relevant edge). Likewise for paths from 11 to right-clique nodes — those go 1→4→5→...1 \to 4 \to 5 \to \ldots, not through 11-22. So b(1-2)=1b(1\text{-}2) = 1 (just the direct (1,2)(1,2) pair, possibly fractional if there are tied paths through other clique members — for K4K_4, between 11 and 22 there are 3 shortest paths counted at length 1 vs length 2: actually shortest is length 1 unique — so b(1-2)=1b(1\text{-}2) = 1).

Girvan-Newman first move: remove edge 44-55 (highest betweenness, splitting the graph into the two natural communities).

A3. Edges: {1-2,1-3,2-3,3-4,4-5,4-6,5-6}\{1\text{-}2, 1\text{-}3, 2\text{-}3, 3\text{-}4, 4\text{-}5, 4\text{-}6, 5\text{-}6\}. m=7m = 7, 2m=142m = 14.

Degrees: k1=2,k2=2,k3=3,k4=3,k5=2,k6=2k_1 = 2, k_2 = 2, k_3 = 3, k_4 = 3, k_5 = 2, k_6 = 2. Check: ∑=14\sum = 14 ✓.

Partition C1={1,2,3},C2={4,5,6}C_1 = \{1, 2, 3\}, C_2 = \{4, 5, 6\}:

- C1C_1 internal edges: {1-2,1-3,2-3}\{1\text{-}2, 1\text{-}3, 2\text{-}3\} → L1=3L_1 = 3. D1=2+2+3=7D_1 = 2 + 2 + 3 = 7.
- C2C_2 internal edges: {4-5,4-6,5-6}\{4\text{-}5, 4\text{-}6, 5\text{-}6\} → L2=3L_2 = 3. D2=3+2+2=7D_2 = 3 + 2 + 2 = 7.

Q=[37−(714)2]+[37−(714)2]=2·(0.4286−0.25)=2·0.1786≈0.357 Q = \left[\frac{3}{7} - \left(\frac{7}{14}\right)^2\right] + \left[\frac{3}{7} - \left(\frac{7}{14}\right)^2\right] = 2 \cdot (0.4286 - 0.25) = 2 \cdot 0.1786 \approx 0.357

Q≈0.357Q \approx 0.357 **— strong community structure**, as expected for two triangles connected by a bridge.

A4. (a) Build the matrices. n=4n = 4. Edges {1-2,2-3,3-4}\{1\text{-}2, 2\text{-}3, 3\text{-}4\}. Degrees k1=1,k2=2,k3=2,k4=1k_1 = 1, k_2 = 2, k_3 = 2, k_4 = 1.

A=(0100101001010010),D=(1000020000200001) A = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}, \quad D = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 \\ 0 & 0 & 2 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}

L=D−A=(1−100−12−100−12−100−11) L = D - A = \begin{pmatrix} 1 & -1 & 0 & 0 \\ -1 & 2 & -1 & 0 \\ 0 & -1 & 2 & -1 \\ 0 & 0 & -1 & 1 \end{pmatrix}

(b) L·**1**L \cdot \mathbf{1} **row-by-row:** 1−1+0+0=01-1+0+0=0; −1+2−1+0=0-1+2-1+0=0; 0−1+2−1=00-1+2-1=0; 0+0−1+1=00+0-1+1=0. So L·**1**=(0,0,0,0)TL \cdot \mathbf{1} = (0, 0, 0, 0)^T ✓.

(c) Symmetry: entries L12=−1=L21L_{12} = -1 = L_{21}, L23=−1=L32L_{23} = -1 = L_{32}, L34=−1=L43L_{34} = -1 = L_{43}, all other off-diagonals are 0 = their transpose. ✓.

(d) Why λ2>0\lambda_2 > 0. The graph is *connected* (single component). The multiplicity of eigenvalue 0 equals the number of connected components — which is 1. So λ1=0\lambda_1 = 0 has multiplicity 1, and λ2\lambda_2 must be strictly positive.

A5. (a) Sign threshold: A={1,2,3}A = \{1, 2, 3\} (positive entries), B={4,5,6}B = \{4, 5, 6\} (negative entries).

(b) Median threshold: sort v2v_2: {−0.5,−0.4,−0.2,0.1,0.4,0.6}\{-0.5, -0.4, -0.2, 0.1, 0.4, 0.6\}. Median is between −0.2-0.2 and 0.10.1, i.e. −0.05-0.05. Split at ≤−0.05\le -0.05 vs >−0.05> -0.05: B={4,5,6}B = \{4, 5, 6\}, A={1,2,3}A = \{1, 2, 3\} — same answer here.

(c) When the entries of v2v_2 have a clear sign change (as here), sign-thresholding is more reliable because it is directly tied to the algebraic decomposition. Median-thresholding is a hack to force balanced cluster sizes; use it only if sign-thresholding produces wildly unbalanced clusters (e.g. one cluster of 1 node).

A6. C4C_4 has n=4n = 4 nodes and edges {1-2,2-3,3-4,4-1}\{1\text{-}2, 2\text{-}3, 3\text{-}4, 4\text{-}1\}. All degrees are 2.

L=(2−10−1−12−100−12−1−10−12) L = \begin{pmatrix} 2 & -1 & 0 & -1 \\ -1 & 2 & -1 & 0 \\ 0 & -1 & 2 & -1 \\ -1 & 0 & -1 & 2 \end{pmatrix}

For cycle CnC_n: λk=2−2cos(2π(k−1)/n)\lambda_k = 2 - 2\cos(2\pi(k-1)/n). With n=4,k=2n = 4, k = 2: λ2=2−2cos(π/2)=2−0=2\lambda_2 = 2 - 2\cos(\pi/2) = 2 - 0 = 2. Note λ2\lambda_2 has multiplicity 2 in C4C_4.

Fiedler vector: v2[i]=cos(2π(i−1)/n)v_2[i] = \cos(2\pi(i-1)/n) for n=4n = 4: v2=(1,0,−1,0)v_2 = (1, 0, -1, 0). (Or by symmetry the orthogonal sin\sin-version gives the second copy.) Sign-threshold: {1},{3}\{1\}, \{3\} separately, with nodes 2 and 4 ambiguous (zero entries). The 4-cycle has a 2-fold symmetry — there are *two* equally good 2-cuts ({1,2}\{1, 2\} vs {3,4}\{3, 4\} and {2,3}\{2, 3\} vs {4,1}\{4, 1\}), reflected by the eigenvalue multiplicity.

A7. Edge betweenness b(e)b(e) = number of shortest paths between all node pairs that pass through edge ee. Edges that lie *between* communities have high betweenness because every shortest path crossing the community boundary must traverse one of the few bridge edges, concentrating shortest-path traffic on them. Internal edges, by contrast, carry only paths within their community plus a small share of cross-community traffic, so their betweenness is low.

A8. Recomputation is required because removing an edge changes the graph's shortest-path structure: paths that previously used the removed edge are now rerouted, redistributing betweenness across the surviving edges. **Example:** in a graph {A-B,B-C,A-C,B-D,D-E,D-F}\{A\text{-}B, B\text{-}C, A\text{-}C, B\text{-}D, D\text{-}E, D\text{-}F\}, edge B-DB\text{-}D is the bridge between {A,B,C}\{A,B,C\} and {D,E,F}\{D, E, F\}. Without recomputation, after removing B-DB\text{-}D you might next remove D-ED\text{-}E or D-FD\text{-}F (which had high original betweenness as the only links to EE and FF); WITH recomputation, after B-DB\text{-}D is gone, the betweennesses of D-ED\text{-}E and D-FD\text{-}F drop because they are now "leaves" of the right community, and you correctly attack the next true bridge.

A9. Modularity:

Q=12m∑ij[Aij−kikj2m]δ(ci,cj) Q = \frac{1}{2m} \sum_{ij}\left[A_{ij} - \frac{k_i k_j}{2m}\right]\delta(c_i, c_j)

The term A_{ij} is the actual edge count between i and j. The term $k_i k_j / (2m)$ is the **expected** edge count between i and j in the **configuration null model** — a random graph in which each node i keeps its degree k_i but its k_i "edge stubs" are rewired uniformly at random to other stubs. Under that null model, the chance that any one of i's stubs lands on j is $k_j / (2m)$, so the expected number of edges between i and j is $k_i \cdot k_j / (2m)$. Thus $A_{ij} - k_i k_j/(2m)$ is the *excess* edges over chance for the pair (i,j).

A10. (1) Symmetric — because D is diagonal (hence symmetric) and A is symmetric (undirected graph), so $L = D - A$ is symmetric. **(2) Positive semi-definite** — $x^T L x = \sum_{(i,j)\in E}(x_i - x_j)^2 \ge 0$, so all eigenvalues ≥ 0. **(3) Smallest eigenvalue is 0** — substitute $x = \mathbf{1}$; each row of L sums to 0, so $L\mathbf{1} = \mathbf{0}$, giving eigenvalue 0 with eigenvector $\mathbf{1}$.

A11. The smallest eigenvalue $\lambda_1 = 0$ has eigenvector $v_1 = \mathbf{1}$ — every entry is the same, so thresholding by sign gives every node the same label and produces no cut. The Fiedler vector v_2 (eigenvalue λ_2) is the smallest *informative* eigenvector: it minimises $x^T L x = \sum (x_i - x_j)^2$ subject to being orthogonal to $\mathbf{1}$ and unit-norm, which is exactly the **continuous relaxation of the minimum cut** problem. Therefore its sign pattern approximately solves the discrete bipartition problem.

A12. (B). $L = D - A$ is symmetric (sum of two symmetric matrices, since both D and A are symmetric, with appropriate signs). Positive semi-definite by the sum-of-squares argument $x^T L x = \sum (x_i - x_j)^2 \ge 0$. Smallest eigenvalue 0 because $L\mathbf{1} = \mathbf{0}$.

A13. (C). *Most likely* (not always) — a bridge between communities accumulates the most shortest paths, but in pathological graphs an internal edge could rival it. The whole point of GN is that high betweenness is a strong heuristic for bridges, not a guarantee.

A14. True. The multiplicity of the eigenvalue 0 in the Laplacian equals the number of connected components. Each component contributes its own indicator vector as an eigenvector with eigenvalue 0. With 3 components there are 3 zero eigenvalues, and the corresponding 3-dimensional eigenspace is spanned by the indicator vectors of the components.

A15. (B). k-way spectral clustering treats each row of the matrix $V = [v_1 \mid v_2 \mid \cdots \mid v_k]$ as a k-dimensional embedding of node i, then runs k-means on these row-vectors to assign each node to one of k clusters. (A) is correct only for $k = 2$ with the Fiedler vector specifically. (C) and (D) are not how spectral clustering works.

§12 — Ending Key Notes (Revision Cards)

Term	Quick-fact
Community	Subset of nodes densely connected internally, sparsely externally.
Edge betweenness $b(e)$	Number of shortest paths (over all source-target pairs) passing through edge e.
Bridge	Edge whose removal disconnects the graph or its current component.
Brandes' algorithm	Compute betweenness with one BFS per source; total $O(nm)$ for unweighted graphs.
BFS forward pass	Label each node with $\sigma(v)$ = number of shortest paths from source to v.
BFS backward pass	Propagate credit from leaves up; flow on (u,v) = $credit(v)\cdot\sigma(u)/\sigma(v)$
Girvan-Newman	Repeatedly remove highest-betweenness edge; recompute after each removal.
Recompute betweenness	NON-NEGOTIABLE between GN steps; structure changes after each removal.
GN complexity	$O(nm^2)$ — feasible $\sim 10^4$ edges.
Dendrogram	Tree of nested splits. Pick level by maximising Q.
Modularity Q	$Q = \frac{1}{2m}\sum_{ij}[A_{ij} - k_i k_j/(2m)]\delta(c_i, c_j)$. Range $[-0.5, 1]$.
Block-form Q	$Q = \sum_c[L_c/m - (D_c/2m)^2]$. Easier to compute.
Configuration null model	Random rewiring preserving degrees. Expected edge count $k_i k_j/(2m)$.
Adjacency matrix A	$A_{ij} = 1$ if edge, else 0. Symmetric for undirected graphs.
Degree matrix D	Diagonal, $D_{ii} = k_i$.
Laplacian L	$L = D - A$. Symmetric, PSD, $L\mathbf{1} = \mathbf{0}$.
$x^T L x$	$\sum_{(i,j)\in E}(x_i - x_j)^2$ — quadratic form, gives PSD.
Trivial eigenpair	$\lambda_1 = 0$, $v_1 = \mathbf{1}$. Useless for clustering.
Algebraic connectivity	λ_2. Small ⇔ graph easy to cut. Multiplicity of 0 = number of components.
Fiedler vector	Eigenvector of λ_2. Sign-threshold gives 2-way cut.
Cheeger inequality	$\lambda_2/2 \le h(G) \le \sqrt{2 \lambda_2 k_{\max}}$.
Spectral 2-way cut	Compute L, find v_2, partition by sign of entries.

Term	Quick-fact
Spectral kk-way cut	First kk eigenvectors → embed → k-means.
Spectral complexity	Dense O(n3)O(n^3); sparse Lanczos O(mm)O(m\sqrt{m}).
GN vs Spectral	GN: hierarchy + interpretability. Spectral: speed + global objective.
Mistake — node vs edge	GN uses EDGE betweenness, not node betweenness.
Mistake — Laplacian sign	L=D−AL = D - A, not A−DA - D. With wrong sign, eigenvalues flip.
Mistake — Fiedler index	v2v_2 uses SECOND-smallest λ2\lambda_2. Skip the trivial λ1=0\lambda_1 = 0.
Mistake — recompute	Must recompute betweenness after every GN edge removal.

§13 — Formula & Algorithm Reference

Concept	Formula	When to use
Edge betweenness	b(e)=∑s≠tσst(e)/σstb(e) = \sum_{s \neq t} \sigma_{st}(e) / \sigma_{st}	Defining the GN edge-removal score.
Brandes — shortest path count	σ(v)=∑u∈parents(v)σ(u)\sigma(v) = \sum_{u \in \text{parents}(v)} \sigma(u)	Forward pass labelling.
Brandes — credit propagation	flow(u,v)=credit(v)·σ(u)/σ(v)\text{flow}(u, v) = \text{credit}(v) \cdot \sigma(u) / \sigma(v)	Backward pass distributing path credit to edges.
Modularity (sum form)	Q=12m∑ij[Aij−kikj/(2m)]δ(ci,cj)Q = \frac{1}{2m} \sum_{ij}[A_{ij} - k_i k_j/(2m)]\delta(c_i, c_j)	Definition; understanding null model.
Modularity (block form)	Q=∑c[Lc/m−(Dc/2m)2]Q = \sum_c [L_c / m - (D_c / 2m)^2]	Hand-computation when partition is given.
Adjacency matrix	Aij=1A_{ij} = 1 if (i,j)∈E(i, j) \in E else 00	Encoding the graph.
Degree matrix	Dii=kiD_{ii} = k_i (degree); Dij=0D_{ij} = 0 otherwise	Building LL.
Graph Laplacian	L=D−AL = D - A	Spectral clustering.
Quadratic form	xTLx=∑(i,j)∈E(xi−xj)2x^T L x = \sum_{(i,j) \in E} (x_i - x_j)^2	Showing L⪰0L \succeq 0; relating to cuts.
Trivial eigenpair	L**1**=**0**L \mathbf{1} = \mathbf{0}, so λ1=0\lambda_1 = 0, v1=**1**v_1 = \mathbf{1}	Confirming computation.
Number of zero eigenvalues	mult(λ=0)(\lambda = 0) = number of connected components	Diagnostic for disconnectedness.
Spectral 2-cut	ci=sign(v2[i])c_i = \text{sign}(v_2[i])	2-way community detection.
Spectral kk-cut	k-means on rows of V=[v1|⋯|vk]V = [v_1 \mid \cdots \mid v_k]	kk-way community detection.
Path graph PnP_n eigenvalues	λk=2−2cos((k−1)π/n)\lambda_k = 2 - 2\cos((k-1)\pi/n)	Closed-form for path graphs.
Cycle graph CnC_n eigenvalues	λk=2−2cos(2π(k−1)/n)\lambda_k = 2 - 2\cos(2\pi(k-1)/n)	Closed-form for cycle graphs.
Cheeger inequality	λ2/2≤h(G)≤2λ2kmax\lambda_2 / 2 \le h(G) \le \sqrt{2 \lambda_2 k_{\max}}	Justifying spectral clustering.

Algorithmic complexity:

- **Brandes' algorithm:** one BFS per source, total O(nm)O(nm) time, O(n+m)O(n + m) space.
- **Girvan-Newman:** mm removals × Brandes per removal = O(nm2)O(n m^2).
- **Spectral (dense):** O(n3)O(n^3) via full eigendecomposition.
- **Spectral (sparse Lanczos):** O(mm)O(m\sqrt{m}) for the bottom-kk eigenvalues, suitable for graphs with ∼106\sim 10^6 edges.

Connections to other weeks:

- **W08-09 — PageRank:** also an eigenvector-based method on graphs. PageRank uses the principal eigenvector of the column-stochastic transition matrix; spectral clustering uses bottom eigenvectors of the Laplacian.
- **W12 — HITS, Topic-sensitive PageRank:** more eigenvector methods. Hubs and authorities come from the eigenvectors of ATAA^T A and AATA A^T — relatives of LL.
- **MMDS Chapter 10:** the canonical reference, especially §10.2 (Girvan-Newman + modularity) and §10.4 (spectral methods, Cheeger).
- **MapReduce / large-scale graphs (W14):** Brandes' algorithm parallelises across BFS source nodes, mapping naturally onto MapReduce. Spectral clustering's eigensolve maps onto distributed Lanczos.

End of W16 Community Detection exam-prep document.